

Pipelines reproducibles del campo al resultado con targets

DE LA TARJETA SD A LA TABLA · ECOACÚSTICA Y
FOTOTRAMPEO

Guillermo Fandos — Universidad Complutense de Madrid · gfandos@ucm.es

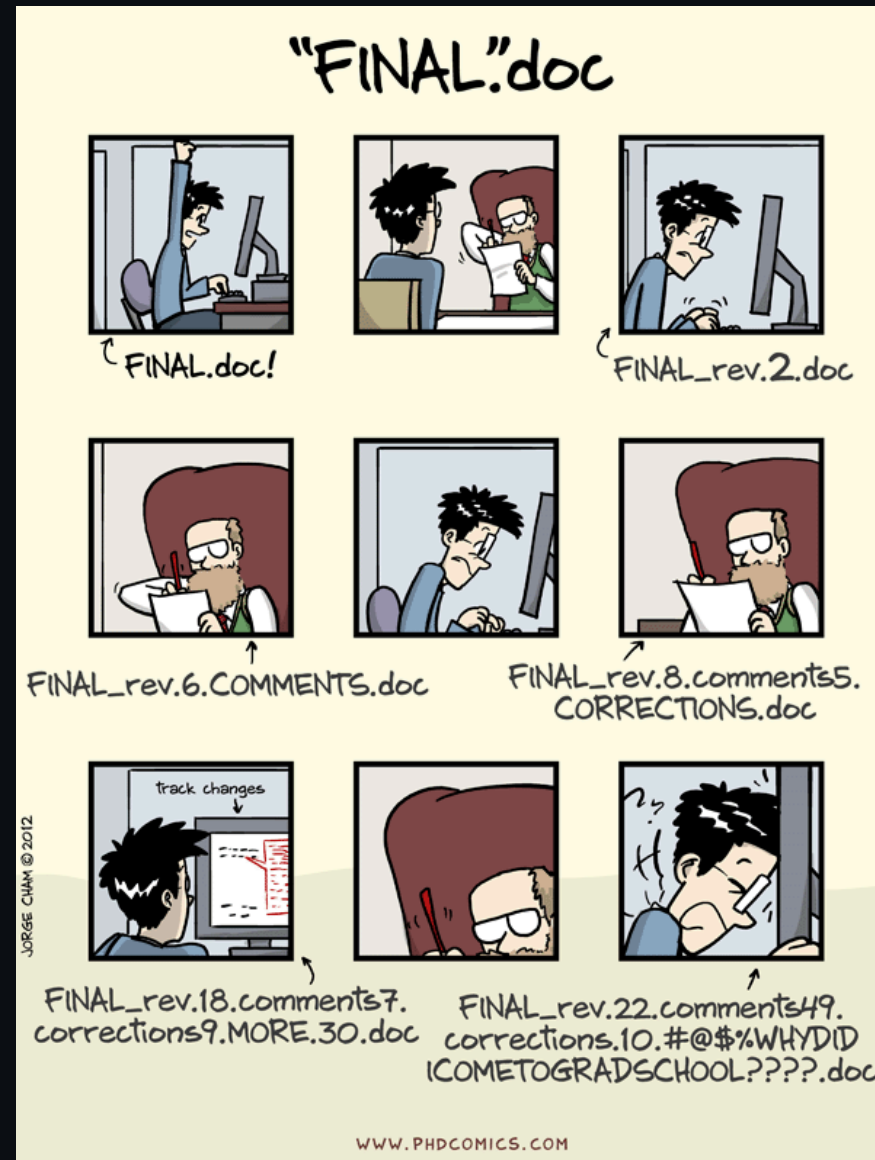
II Jornadas de Ecoinformática de la AEET · Fundación Biodiversidad,
Sevilla

1 de octubre de 2026



¿Te suena esto?

— EL PROBLEMA · 0:00



«Sálvame de mí mismo»

— EL PROBLEMA · 0:01



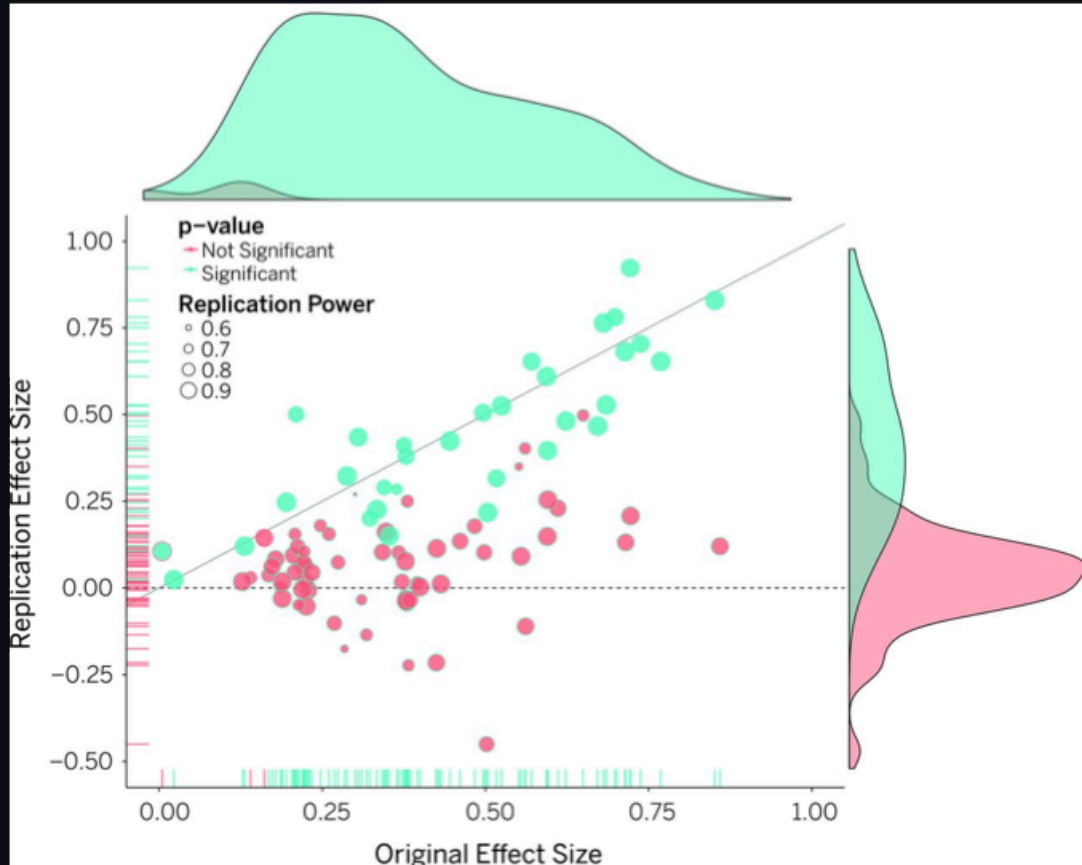
Cambia un dato o un umbral.
¿Qué hay que volver a correr,
y en qué orden?

La solución no es recordarlo mejor. Es que lo recuerde el ordenador.

Tweet: Brianna (@fossilosophy), 21 de febrero de 2018.

La crisis de reproducibilidad

– TAMBIÉN EN ECOLOGÍA • 0:02



Ecología: 346 artículos en 14 revistas **con política de código.**

- Solo el 27 % compartía el código.
- Solo el 21 % tenía datos **y** código completos.
- Y según los autores, probablemente es una sobreestimación.

Culina et al. (2020, *PLOS Biology*), doi:10.1371/journal.pbio.3000763

Psicología: solo el 36 % de 100 estudios replicó (Open Science Collaboration 2015, *Science*).

El análisis de datos como Sísifo

– SCRIPTS SECUENCIALES = SUBIR LA ROCA CADA VEZ · 0:04



- Cambias el **umbral de QC** → ¿qué scripts hay que volver a correr?
- Se incorpora **alguien nuevo al equipo** → ¿en qué orden se ejecuta todo?
- **Revisión del artículo** → ¿de dónde sale exactamente esa tabla?
- **Tres temporadas de campo** → ¿qué versión del código la generó?

La solución no es recordar mejor. Es **no tener que recordar**.

François, Tumba de Carlo Ruspi (1825) · dominio público

El caso de hoy: de la SD a la tabla

– ECOACÚSTICA PRIMERO; FOTOTRAMPEO DESPUÉS · 0:05

WAV CRUDOS | METADATOS | QC | ÍNDICES | TABLA

LOS DATOS

2 grabadores AudioMoth · 6 WAV de 10 s · 1 archivo truncado · 1 hueco de calendario · `sites.csv` con el diseño de muestreo

EL RESULTADO

`output/tabla_analisis.csv`: una fila por grabación válida, con índices acústicos, metadatos y sitio. Lista para modelar.

Mismo patrón en fototrampeo, telemetría o capas ráster: **muchos archivos crudos → pasos por archivo → una tabla.**

¿Qué es **targets**?

– BLOQUE 1 · LA IDEA · 0:06

Un **Make para R**: tú escribes los pasos; **targets** decide **qué ejecutar y en qué orden**.

1. **Lee tu código** y ve qué paso usa el resultado de cuál: con eso construye el grafo de dependencias.
2. **Guarda cada resultado** en `_targets/`, con una huella (*hash*) de su código, sus datos y sus dependencias.
3. **En cada `tar_make()` compara las huellas**: si nada cambió, se salta el paso; si algo cambió, recalcula ese paso y todo lo que depende de él, en orden.

El análisis está siempre al día y cualquiera lo reproduce con **un solo comando**.

Landau (2021, *JOSS*), doi:10.21105/joss.02959

De scripts a funciones

– BLOQUE 1 · LO QUE CAMBIA EN TU CÓDIGO

```
# Antes: analisis.R, de arriba abajo
files <- list.files("data_raw", recursive = TRUE)
meta <- ...           # 40 líneas de parseo
meta$qc <- ...        # 30 líneas de QC
idx <- ...            # 50 líneas de índices
tab <- merge(idx, meta) |> merge(sites)
write.csv(tab, "output/tabla.csv")
```

```
# Después: R/qc.R, una función por paso
qc_flag <- function(meta, expected_dur_s,
                     tol_frac = 0.05) {
  ...
}

# _targets.R, un target por función
tar_target(meta_qc,
           qc_flag(meta_raw, expected_dur_s))
```

Una función = un paso = un `tar_target()`. No reescribes el análisis: lo partes y le pones nombre.

La anatomía de un `tar_target()`

– BLOQUE 1 · UN TARGET REAL DE ESTE PIPELINE

```
tar_target(  
  name      = meta_raw,           # nodo del grafo y objeto en caché  
  command   = parse_one(audio_files, # otro target: dibuja una flecha  
                    tz = recorder_tz), # variable de _targets.R: también es dependencia  
  pattern   = map(audio_files)    # opcional: una rama por archivo  
)
```

- Los **símbolos del comando** son las dependencias: si cambia `audio_files`, `recorder_tz` o la función `parse_one()`, se recalcula.
- El resultado se lee de la caché con `tar_read(meta_raw)` o `tar_load(meta_raw)`.
- `format = "file"`: el valor es una ruta y `targets` vigila el archivo.
- `pattern = map()`: ramas dinámicas, una por archivo (bloque 2).

El ciclo de trabajo

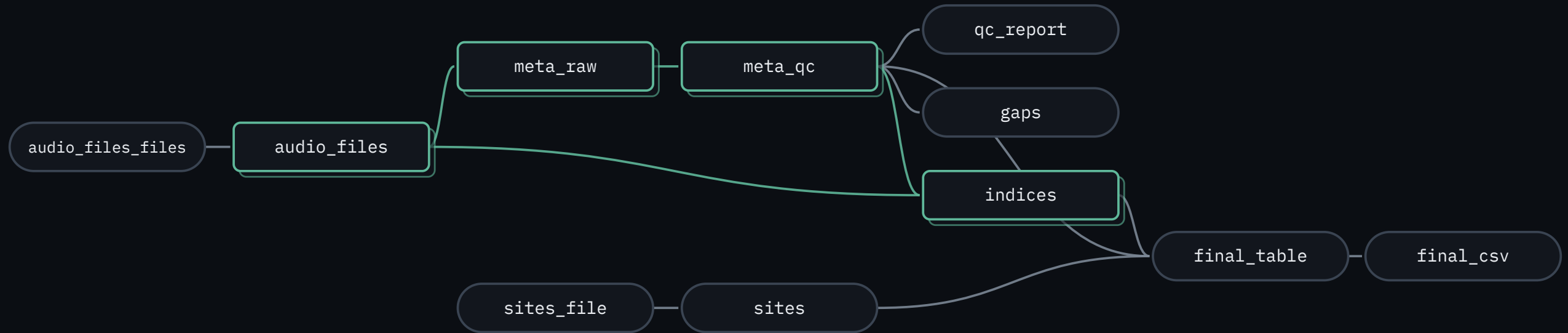
– BLOQUE 1 · EL DÍA A DÍA



No hay que acordarse de qué volver a correr: `tar_make()` lo decide cada vez.

El grafo se deduce del código

– ESTE PROYECTO · 11 TARGETS



Rectángulos apilados = targets **con ramas** (una por archivo); flechas verdes = rama a rama. Óvalos = targets normales.
`tar_visnetwork()` lo dibuja interactivo y coloreado según lo que está desactualizado.

En vivo: un solo comando

– DEMO 1 · 0:12

EN VIVO · DEMO 1

```
targets::tar_manifest()      # qué targets hay
targets::tar_visnetwork()    # el grafo
targets::tar_make()          # ejecuta todo
targets::tar_read(gaps)      # lee un resultado de la caché
targets::tar_make()          # otra vez: ✓ skipped pipeline
```

Ejercicio 1 · 3 min · `practica.R`: ejecuta el pipeline, lee la tabla (5 filas) y el hueco de AM01. ¿Cuántas ramas tiene cada etapa? `tar_progress_branches()`

Una rama por archivo

BLOQUE 2 · ESCALAR · 0:17

Un fallo silencioso: la SD nueva que nadie ve

– BLOQUE 2 · EL PROBLEMA

```
tar_target(audio_files, list_audio("data_raw")) # lista los WAV de la carpeta
```

LO QUE TARGETS VIGILA

el código del comando y de sus funciones
(`list_audio()`) · los objetos que usa (el texto
"data_raw") · los archivos declarados con `format`
= "file"

LO QUE NO VIGILA

lo que hay **dentro** de una carpeta · un archivo que
lees sin declararlo

Copias WAV nuevos → nada de lo vigilado cambia → `audio_files` se da por actualizado
y **no se vuelve a listar la carpeta:**

```
✓ skipped pipeline [11 skipped] # la tabla sale igual que antes
```

Sin error ni aviso: los archivos nuevos no entran en el análisis.

La solución: seguir los archivos y ramificar

– TARCHETYPES + DYNAMIC BRANCHING

```
# 1. Vigilar cada archivo de entrada (nuevos Y modificados)
tar_files_input(audio_files, list_audio(audio_dir)),

# 2. Una rama por archivo en cada paso
tar_target(meta_raw, parse_one(audio_files, tz = recorder_tz),
           pattern = map(audio_files)),
tar_target(meta_qc, qc_flag(meta_raw, expected_dur_s),
           pattern = map(meta_raw)),
tar_target(indices, indices_if_pass(audio_files, meta_qc),
           pattern = map(audio_files, meta_qc)),

# 3. Al usar el target entero, las ramas se combinan solas (vctrs)
tar_target(qc_report, qc_summary(meta_qc)),
```

Cada rama tiene su propio hash: un archivo nuevo es una rama nueva; los demás siguen en caché.

tar_files() o tar_files_input()

– UN DETALLE QUE CAMBIA EL COMPORTAMIENTO

	tar_files()	tar_files_input()
Para qué	archivos que genera el pipeline	archivos de entrada (tus datos crudos)
Cuándo lista los archivos	en cada <code>tar_make()</code> (se ejecuta siempre)	cada vez que se lee <code>_targets.R</code>
Archivo nuevo / modificado	lo detecta	lo detecta
<code>tar_outdated()</code> , <code>tar_visnetwork()</code>	marcan como desactualizado todo lo que cuelga de los archivos	dicen la verdad
Muchos archivos	una rama por archivo	<code>batches</code> = agrupa archivos por rama

Con decenas de miles de archivos, cada rama tiene un coste fijo: agrupa con `batches`. Ver el manual de `tarchetypes`.

Ejercicio 2 · llega una SD nueva

– 6 MIN · PRACTICA.R · 0:21

Predice antes de ejecutar: si añades 2 grabaciones de AM01, ¿cuántas ramas de `meta_raw` e `indices` se ejecutan?

```
file.copy(list.files("nueva_descarga/AM01", pattern = "\\*.WAV$", full.names = TRUE),  
          "data_raw/AM01")  
targets::tar_make()  
targets::tar_progress_branches()
```

Después restaura: borra las dos copias y vuelve a ejecutar `tar_make()`.

Ejercicio 2 • resultado

– SOLO LO NUEVO

	name	branches	skipped	completed
1	audio_files	8	6	2
2	indices	8	6	2
3	meta_qc	8	6	2
4	meta_raw	8	6	2

- 2 ramas nuevas por etapa; las 6 anteriores salen de la caché.
- La tabla pasa de 5 a 7 filas. Los resúmenes (`qc_report`, `gaps`, `final_table`) se rehacen porque dependen de todas las ramas.
- Con 3 temporadas de campo, eso es procesar **una SD**, no todo el archivo histórico.

Código y valores

BLOQUE 3 · QUÉ SE RECALCULA DE VERDAD · 0:28

Demo · umbral de QC: predice a mano alzada

– EN VIVO · LO REPITES EN CASA CON PRACTICA.R

En R/qc.R, `tol_frac = 0.05` → `0.75`. Guarda.

¿Cuántas ramas de `meta_raw`, `meta_qc` e `indices` se recalculan? ¿0, 1 o 6?

```
targets::tar_outdated()      # ¿aparece meta_raw?  
targets::tar_make()  
targets::tar_progress_branches()
```

Demo · resultado

– TARGETS COMPARA VALORES, NO SOLO CÓDIGO

	name	branches	skipped	completed
1	audio_files	6	6	0
2	indices	6	5	1
3	meta_qc	6	0	6
4	meta_raw	6	6	0

META_QC · 6 DE 6

Cambió la función `qc_flag()`: todas sus ramas quedan desactualizadas y se ejecutan.

INDICES · 1 DE 6

Solo cambió el **valor** de QC del archivo truncado. Las otras 5 ramas reciben lo mismo que antes y se saltan.

El parseo (`meta_raw`) ni se toca. Con BirdNET en lugar de índices, esto es la diferencia entre minutos y horas.

Otro sensor, el mismo esqueleto

BLOQUE 4 · FOTOTRAMPEO · 0:32

De la tarjeta de la cámara a la tabla

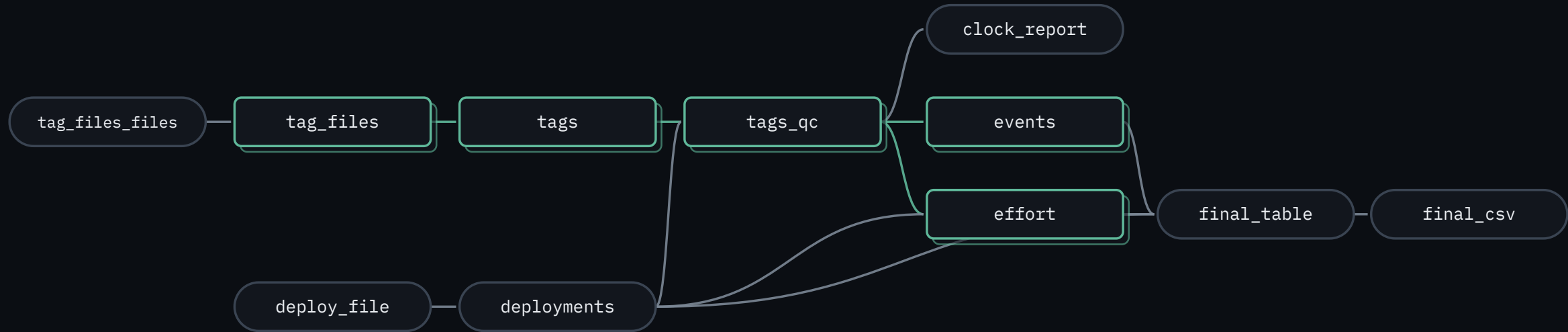
– FOTOTRAMPEO • LO QUE CAMBIA Y LO QUE NO

	Ecoacústica	Fototrampeo
Una rama por...	archivo WAV	tarjeta (<code>etiquetas.csv</code>)
Hora del evento	nombre vs cabecera del WAV	fecha EXIF vs fechas del despliegue
Fallo típico del reloj	zona horaria, deriva	vuelve a 2000-01-01 al cambiar las pilas
Motor externo	BirdNET	detector (p. ej. MegaDetector) + etiquetado
Umbral que decides tú	tolerancia de duración	independencia entre eventos
Diseño de muestreo	<code>sites.csv</code>	<code>despliegues.csv</code> → esfuerzo (cámara-día)

Igual con telemetría GPS (una rama por individuo) o SDM (una rama por especie).

Fototrampeo en **targets**

– SEGUNDO PROYECTO EN EL MISMO REPOSITORIO



```
Sys.setenv(TAR_PROJECT = "fototrampeo") # declarado en _targets.yaml
targets::tar_make()                     # mismo comando, otro pipeline
Sys.unsetenv("TAR_PROJECT")             # volver al caso acústico
```

Cada proyecto tiene su propia caché. Sin `setwd()`: las rutas son relativas a la raíz del repositorio.

En vivo: el reloj y el umbral

– DEMO 2 · FOTOTRAMPEO

EN VIVO · DEMO 2

```
camera_id fotos fuera_despl vacias conf_baja pasan_qc
1      CT01    67           0      8         3       56
2      CT02    88           0      7         4       77
3      CT03    48          26     10         2       18
AVISO: fotos fuera del despliegue en CT03 -> revisa el reloj de la cámara.
```

En fototrampeo/_targets.R: indep_min <- 30 → 60.

```
      name branches skipped completed
1    effort        3        3         0
2   events        3        0         3
3     tags        3        3         0
4 tags_qc         3        3         0
```

Solo se rehacen los eventos. El parseo, el QC y el esfuerzo salen de la caché.

Decisiones que cambian el resultado

– EL PIPELINE LAS HACE VISIBLES, NO LAS TOMA POR TI

UMBRAL DE INDEPENDENCIA

30 → 60 min: los eventos bajan de **64 a 45** (–30 %). 30 min es una convención, no una propiedad biológica.

ESFUERZO DE CT03

El reloj falla: el esfuerzo se corta en la última foto válida (**3,5 días** de 14). Con tan poco esfuerzo, sus tasas son muy inestables.

La tasa de detección (eventos por 100 cámara-día) **no es abundancia**: depende de la detectabilidad, del área de campeo y de la colocación de la cámara (Sollmann et al. 2013, *Biol. Conserv.*).

Trampas frecuentes

– PARA LLEVAR · 0:38

ARCHIVOS un `tar_target()` que lista una carpeta no ve archivos nuevos → `tar_files_input()`

RELOJ compara las fechas con el despliegue, no solo entre sí

SALIDAS si escribes a disco, devuelve la ruta y usa `format = "file"`

GLOBALES los parámetros de `_targets.R` son dependencias: cambiarlos invalida lo de aguas abajo

RAMAS miles de archivos → batches; el coste por rama no es cero

JOINS comprueba que las filas no crecen y que no hay claves sin pareja

WINDOWS el proyecto fuera de OneDrive

Llévate el repositorio

– RECURSOS · 0:39

- **Repo:** github.com/guifandos/targets-campo-resultado
- `practica.R`: la demo del umbral y extras para casa (zona horaria, join que infla filas, fototrampeo)
- `docs/recursos.md`: paralelizar con `crew`, informes con `tar_quarto()`, herramientas de ecoacústica y fototrampeo
- **Mañana, 15:00:** taller de ciencia reproducible de Ignasi Bartomeus

Gracias. ¿Preguntas?



Imágenes: fotos de campo del autor · cámara trampa y zorro: Dodro (CC BY 4.0, Wikimedia Commons) · siluetas: PhyloPic (CC0; zorzal: Zeynel Cebeci, CC BY 4.0) · tira: Jorge Cham, *PhD Comics* · tweet: Brianna (@fossilosophy)

